Geophysical Journal International

Geophys. J. Int. (2025) **243,** 1–19
Advance Access publication 2025 July 31
Magnetic and Electrical Fields

https://doi.org/10.1093/gji/ggaf290

Evaluation of an iterative framework for geophysical electromagnetic forward and inverse modelling problems

Michael Weiss *, Raphael Rochlitz and Thomas Günther†

LIAG Institute of Applied Geophysics, 30655 Hanover, Germany. E-mail: weissmilu@gmail.com,Raphael.Rochlitz@liag-institut.de, Thomas.Guenther@geophysik.tu-freiberg.de

Accepted 2025 July 24. Received 2025 July 15; in original form 2024 December 10

SUMMARY

Modelling and inversion of controlled-source electromagnetic data requires elaborate numerical tools. The major challenge is the high computational cost of computing solutions to numerous forward problems (for the forward responses as well as the sensitivity matrix). Forward modelling is accomplished using either a direct or an iterative solver. Current modelling suites predominantly employ direct solution methods in the forward operator since multiple solutions are easily accessible using inexpensive and quick forward-backward substitution after an initial resource-demanding matrix factorization step. Iterative techniques, on the other hand, require little resources for single forward solutions, and are yet very time consuming if solutions for many right-hand sides are to be computed. Evaluations of different solution techniques for modelling and inverse problems are only sparsely investigated. In light of this, we integrated an iterative solver as alternative in the forward and inversion operators of the open-source software custEM and pyGIMLi. In particular, we implemented a two-level iterative scheme where the outer solver employs a generalized conjugate residual algorithm pre-conditioned with a highly efficient block-based pre-conditioner for square blocks. The inner-level solver is either of the same type as the outer solver, but pre-conditioned with the auxiliary-space Maxwell pre-conditioner, or may alternatively be a direct solver. In this paper, we evaluate the described iterative forward operator for forward modelling tasks for the Marlim R3D model for a single as well as numerous right-hand side vectors and compare the performance to the direct solver MUMPS. We further investigate the solver's applicability on small and medium-sized computing platforms. We then examine the iterative solver for inversions of synthetic land-based and semi-airborne data in terms of computational requirements. Our results demonstrate that forward modelling tasks are best performed using an iterative approach for single source problems. Moreover, simulations of large and complex problems are accessible even on small computing platforms such as laptops in very reasonable time. For inversions, the iterative forward operator, in particular the mixed iterative-direct-based one, performs equally well in terms of time as the direct one while reducing the memory demands for the computations of the forward responses and the data sensitivities.

Key words: Controlled source electromagnetics; Inverse theory; Numerical modelling.

1 INTRODUCTION

Geophysical electromagnetic (EM) techniques aim at imaging and inferring the electrical and magnetic properties of the shallow and deep subsurface of the Earth of a target region. Electromagnetic surveys commonly proceed by acquiring data which then is processed, inverted for and lastly interpreted in the context of the existing geological and geophysical knowledge. All these steps, that is data collection (see e.g. Chave & Jones 2012), data processing (see e.g. Egbert 1997; Myer *et al.* 2011; Streich *et al.* 2013; Chave 2017; Becken *et al.* 2020) and forward/inverse modelling (see e.g. Avdeev 2005; Commer & Newman 2008; Börner 2010; Siripunvaraporn 2012; Newman 2014; Wang *et al.* 2021), come with their own sets of challenges, which have been discussed in details in the above-mentioned publications and references therein.

From a computational point of view, the modelling and inversion stage represents the main bottleneck due to the associated high computational costs which may limit the size of feasible problems

Now at: Sveriges Geologiska Undersökning, 751 28 Uppsala,Sweden.
†*Now at:* TU Bergakademie Freiberg, 09599 Freiberg, Germany.

© The Author(s) 2025. Published by Oxford University Press on behalf of The Royal Astronomical Society. This is an Open Access article distributed under the terms of the Creative Commons Attribution License (https://creativecommons.org/licenses/by/4.0/), which permits unrestricted reuse, distribution, and reproduction in any medium, provided the original work is properly cited.

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

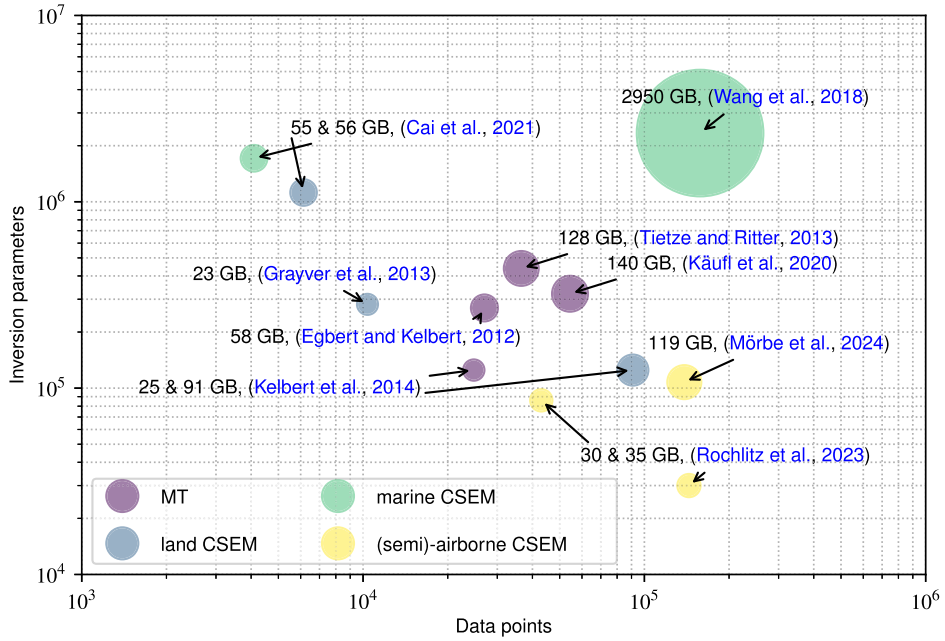


Figure 1. Overview of the dimensions of EM data sets. The size of the circle indicates the storage requirement for the full sensitivity matrix in Gigabytes (GB) assuming each entry requires 8 bytes storage. Note that the legend size circles correspond to 100 GB.

depending on the available computing platform. Although accessibility and performance of all types of computing platforms have increased greatly, it can still pose the main restriction to invert very large data sets due to the large memory requirements. Fig. 1 gives a summary of the size of EM inversion problems and the associated memory demands for the storage of the sensitivity matrix if they were computed explicitly. It is noted that there exists inversion algorithms employing techniques that avoid the explicit computation of the sensitivities and for which the main memory cost is incurred by matrix factorizations instead of the storage of the sensitivity matrix (see e.g. Mackie & Madden 1993; Newman & Alumbaugh 2000; Rodi & Mackie 2001; Egbert & Kelbert 2012; Grayver *et al.* 2013). Apart from the high memory requirements of the storage of the Jacobian and/or matrix factorizations, solving the related forward problems for multiple sources at several frequencies as well as the required numerous forward solutions to compute data sensitivities for inverse problems is time intensive and adds to the high resource requirements of EM modelling.

Forward modelling may be accomplished in time or frequency domain using direct or iterative solvers which have their own advantages and disadvantages. The literature in EM geophysical modelling is rich in publications detailing the different discretization approaches used for forward modelling. However, only a few comparative studies discuss the solution accuracy of complex real world model between different modelling codes (e.g. Werthmüller *et al.* 2021; Castillo-Reyes *et al.* 2023) or evaluate the performance, resource requirements and scaling behaviour of direct and iterative solvers for forward and inverse modelling (e.g. da Silva *et al.* 2012; Kordy *et al.* 2016; Puzyrev *et al.* 2016). Further, such investigations rarely discuss modelling codes in terms of their applicability on different computing platforms which, recalling the high computational requirements and thus dependence on sufficiently powerful platforms, might be of interest to the end user.

The modelling codes and software in the field of EM modelling, whether available as open source software or on demand for academic purposes, have access to direct and iterative solution techniques, as summarized in the limited overview in Table A1 in Appendix A. It is remarked that proprietary codes are excluded from this overview as it is unclear which solvers they might have available. Nonetheless, there exist only few evaluations for forward modelling tasks between different codes and within the same environment. For inverse problems, there are hardly any comparisons of solution methods between codes or within the same environment. One major reason for that is the inherent difficulty associated with comparing different solution methods as one needs to navigate and control many variables such as the respective forward and inverse modelling algorithms, the employed discretization method, the mesh discretization, the parallelization techniques and the computer architecture. Solver comparisons do thus have to account for a variety of factors, which makes cross-code evaluations difficult. However, solution method comparisons can be accomplished within the same code framework as comparisons are not afflicted by the aforementioned variables with the exception of the solver implementations. Hence, most variables are accounted for, thus allowing for an as fair as possible and unbiased comparison between a direct and iterative solution technique, which in addition does not depend on the used modelling framework. Thus, analysing the scope of optimal solution strategies in EM forward and inverse modelling would benefit greatly from investigating both direct solvers and iterative solvers in the same environment. To enable such an assessment, the toolbox custEM (Rochlitz *et al.* 2019) is extended by incorporating an iterative framework based on the robust and efficient block-based PREconditioner for Square Blocks (PRESB) (Weiss *et al.* 2023). Hence, custEM provides ideal pre-requisites for a one-to-one comparison of different solution techniques, namely of the direct solver MUMPS with the PRESB-pre-conditioned iterative solver, for EM modelling applications. The main objective of this paper is to present a comprehensive comparison of the two solution approaches for both forward and inverse modelling based on performance criteria such as resource requirements and scaling behaviour. A secondary goal is to outline the best strategies in selecting the most suitable method guided by the aforementioned measures. In

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

addition, cross-platform applicability of the two solvers is investigated. It is noted that the comparison is performed for CSEM problems only.

The paper is structured as follows: Section 2 briefly describes the modelling environment used in this work, that are the open-source Python-based toolboxes custEM and pyGIMLi (Rücker *et al.* 2017). It further introduces the iterative solver as a forward operator for both forward and inverse modelling tasks. In Section 3, representative models for semi-airborne, land- and marine-based surveys in controlled-source electromagnetic applications used in this work are outlined. Sections 4 and 5 present the results of the modelling comparison for forward and inverse problems, respectively. Section 6 discusses best strategies, benefits, limitations, scope and the applicability of the respective solution method for both forward and inverse modelling problems. Lastly, Section 7 addresses further potential and future challenges and draws conclusions.

2 MODELLING ENVIRONMENT

2.1 custEM and pyGIMLi

We build upon the CSEM modelling and inversion framework presented by Rochlitz *et al.* (2023). The forward solver custEM uses the finite-element method on unstructured tetrahedral meshes. In context of modelling realistic survey geometries and to properly account for exact transmitter layouts on arbitrary topography, it solves the total-field formulation of the Maxwell equations to calculate the electromagnetic fields and the sensitivities. The message-parsing interface (MPI) provides the option of mesh distribution and parallel computing as implemented in the underlying finite-element library FEniCS (Logg *et al.* 2012; Alnaes *et al.* 2015), which serves also to interact with PETSc. Previously to this work, the direct solver MUMPS was the only available solver for the systems of equations. The open-source library pyGIMLi provides solving inverse problems in geophysics with a variety of approaches. It can handle problem-specific model and data transformations as well as joint inversion. We use the Gauss–Newton minimization approach, solving the dense system of equations in the inverse problem with a matrix-free least-squares conjugate gradient solver.

2.2 Iterative solver

The iterative framework described by Weiss *et al.* (2023) was incorporated into custEM for the total-field formulations for both forward and inverse modelling tasks. At the core of the implementation is the package petsc4py (Dalcin *et al.* 2011) that provides an interface to PETSc's functionalities (Balay *et al.* 2024).

In the following, the iterative solver is briefly summarized and the reader is referred to the underlying publication and references therein for a more detailed presentation. Let matrix $\mathcal{A}$ be a real-valued two-by-two block system of general form

$$\mathcal{A} = \begin{bmatrix} \mathbf{A} & -b\,\mathbf{B_2} \\ a\,\mathbf{B_1} & \mathbf{A} \end{bmatrix}, \tag{1}$$

for which it is assumed that block $\mathbf{A}$ is symmetric positive definite and that scalars a and b are non-zero and of the same sign. If the block system $\mathcal{A}$ fulfils the aforementioned conditions, the pre-conditioner known as PRESB short for 'PREconditioner for Square Blocks' (see Axelsson *et al.* 2014), that reads

$$\mathcal{P}_{\text{PRESB}} = \begin{bmatrix} \mathbf{A} & -b\,\mathbf{B}_2 \\ a\,\mathbf{B_1} & \mathbf{A} + \sqrt{ab}(\mathbf{B_1} + \mathbf{B_2}) \end{bmatrix}, \tag{2}$$

can be applied to pre-condition and solve the block system $\mathcal{A}$ efficiently. In particular, it can be proved that all the eigenvalues of the pre-conditioned system $\mathcal{P}^{-1}_{\text{PRESB}}\mathcal{A}$ are real-valued and cluster in the interval 0.5 and 1 (see e.g. Axelsson *et al.* 2016) which translates to excellent convergence properties of this pre-conditioner. In addition, the pre-conditioner is robust with respect to mesh discretization, the scalars a and b and material parameters that enter through the blocks of $\mathcal{A}$ meaning that the pre-conditioner and solver are independent on modelling examples, given that abovementioned matrix properties are fulfilled.

EM problems employing total or secondary E-field or H-field approaches can be written in form of a real-valued block system (1) by splitting the complex fields into real and imaginary parts (cf. Grayver *et al.* 2013; Grayver 2015; Rochlitz *et al.* 2019) and slightly adjusting the formulation (cf Weiss *et al.* 2023). Note that the off-diagonal blocks are equal meaning $\mathbf{B_2} = \mathbf{B_1}^T = \mathbf{B}$ and that the scalars a and b are simply equal to one.

It can be shown that applying the pre-conditioner PRESB is equivalent to solving a system of the form (see Axelsson *et al.* 2017)

$$\begin{bmatrix} \mathbf{D} & -\mathbf{C}^T \\ \mathbf{C} & \mathbf{D} + (\mathbf{C} + \mathbf{C}^T) \end{bmatrix} \begin{bmatrix} \mathbf{w_1} \\ \mathbf{w_2} \end{bmatrix} = \begin{bmatrix} \mathbf{f_1} \\ \mathbf{f_2} \end{bmatrix}, \tag{3}$$

where the blocks $\mathbf{C}$ and $\mathbf{D}$ are the stiffness and mass matrices as defined in Rochlitz *et al.* (2019). The solution of this system (3) is given in Algorithm 1 as

Algorithm 1: Applying the preconditioner PRESB

1. Solve $(\mathbf{D} + \mathbf{C})\,\mathbf{g} = \mathbf{f_1} + \mathbf{f_2}$
2. Compute $\mathbf{Dg}$ and $\mathbf{f_1} - \mathbf{Dg}$
3. Solve $\left(\mathbf{D} + \mathbf{C}^T\right)\mathbf{h} = \mathbf{f_1} - \mathbf{Dg}$
4. Compute $\mathbf{w_1} = \mathbf{g} + \mathbf{h}$ and $\mathbf{w_2} = -\mathbf{h}$

and amounts to solving two linear systems with $\mathbf{D} + \mathbf{C}$ and $\mathbf{D} + \mathbf{C}^T$, one matrix–vector multiplication and three vector operations.

The two linear systems in Algorithm 1 are discretized $\mathcal{H}_0(\text{curl}, \Omega)$ problems which can be solved fast and efficiently using the auxiliary-space technique (see e.g. Xu 1996; Hiptmair & Xu 2007; Kolev & Vassilevski 2009). Here, the auxiliary-space Maxwell solver (AMS) implemented in the library for high performance pre-conditioners and solvers featuring multigrid methods for the solution of large and sparse linear systems of equations called *hypre* (Falgout & Yang 2002) is used to pre-condition an iterative solver to obtain solution to the systems $\mathbf{D} + \mathbf{C}$ and $\mathbf{D} + \mathbf{C}^T$. Alternatively, these systems may be solved using a direct solver such as MUMPS (Amestoy *et al.* 2000).

The resulting block system adopted to EM modelling is pre-conditioned using PRESB and may be solved using a flexible generalized minimal residual (FGMRES; Saad 1993), a generalized conjugate residual (GCR; Eisenstat *et al.* 1983) or a generalized conjugate gradient (GCG; Axelsson & Vassilevski 1991) method. At each iteration, pre-conditioner PRESB is applied which necessitates the operations outlined in Algorithm 1. The iterative algorithm thus consists of an outer solver with a nested inner algorithm in which two systems require solving.

In the incorporated implementation, the GCR method is employed as the outer solver, and the inner systems are either solved iteratively using the FGMRES algorithm pre-conditioned with the auxiliary-space Maxwell pre-conditioner or directly using MUMPS. For clarity, a simplified inversion scheme is shown in Fig. 2, which integrates the iterative solver as the forward operator and indicates where and for what purpose it is called within the inversion.

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

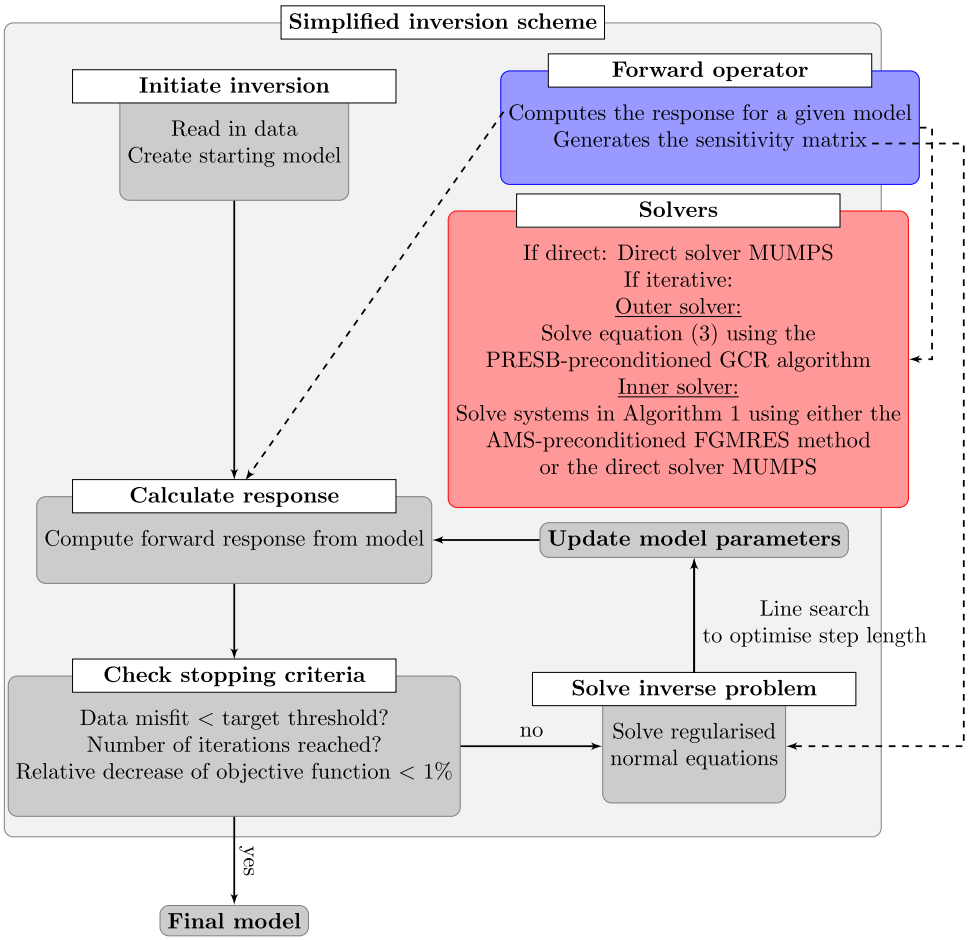


Figure 2. Simplified inversion scheme indicating where and for what purpose the forward operator is employed in the inversion.

3 GEOPHYSICAL MODEL DESCRIPTIONS

In this section, we outline the models selected for the modelling studies. In particular, we choose models that are representative for semi-airborne, land- and marine-based surveys in controlled-source electromagnetic applications. As the focus of this study lies on the comparison between solution methods, we purposefully selected known, tested and discussed models and will solely discuss the results briefly.

3.1 Example 1: the MR3D model

The MR3D model represents a realistic 3-D anisotropic resistivity model derived from seismic and well log data (Carvalho & Menezes 2017). The model is located in the north-eastern part of the Campos Basin, offshore Brazil, on the continental margin, in which sand-rich turbiditic bodies compose the most valuable petroleum reservoirs that constitute possible targets for a CSEM survey. A detailed description of the geology and the derived geoelectric model are given in Carvalho & Menezes (2017) and Correa & Menezes (2019) and references therein. The latter work also released CSEM data for six frequencies for the MR3D model computed using the SBLwiz software from EMGS (Maaø 2007), thus providing a complex and realistic model in the domain of marine CSEM modelling. The simulated CSEM responses comprise data from 500 receivers that are laid out on a regular grid (20 by 25 lines in eastern and northern directions) with a spacing of 1 km and that are located on the seafloor (see Fig. 3a). Source towlines were located along the 25 east–west and 20 north–south lines, 50 m above the receiver grid and source positions were spaced at 100 m along each towline. The computational model for MR3D spans a volume of $56\times51\times6$ km^3 and is discretized using $515\times563\times310$ cells of $100\times100\times20$ m^3 size yielding an finite-difference time-domain mesh with almost 90 million cells.

MR3D and the associated published CSEM responses are intended as a realistic benchmark model to study reservoirs at the Brazilian offshore margin, but are also suited to validate modelling codes on the basis of a complex and realistic model against an industry-proofed code. In a comparative modelling paper by Werthmüller *et al.* (2021) the responses for receiver 04Rx251a for the inline towline 04Tx013a and broadside towline 04Tx014a (see Fig. 3) were reproduced using various modelling codes, including custEM, to compare the modelling responses and to assess the influence of mesh discretization on modelling accuracy.

3.2 Example 2a and 2b: synthetic land-based and semi-airborne CSEM model

The synthetic Schleiz model used here is an adjusted model from the previously presented model in Rochlitz *et al.* (2023) which

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

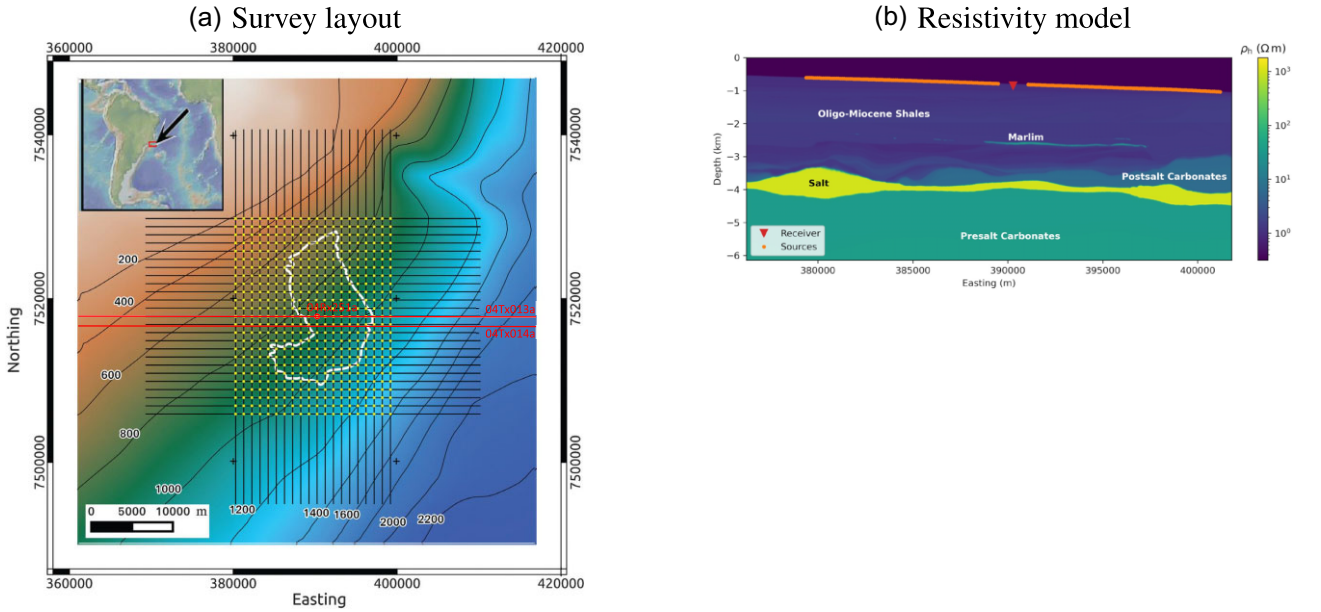


Figure 3. Example 1: Survey layout projected on top of the bathymetry of the MR3D area showing the Marlim reservoir (white), the receiver locations (yellow dots) and the towline grid. In red, the two towlines and the receiver location for the open source data are indicated (see Correa & Menezes 2019). For towline 04Tx013a, the horizontal resistivity model is shown on the left. The vertical transverse isotropy of all the formations, apart from the electrically isotropic air, seawater and salt layer, is $\lambda = \sqrt{2}$. Maps were taken and modified from Werthmüller *et al.* (2021).

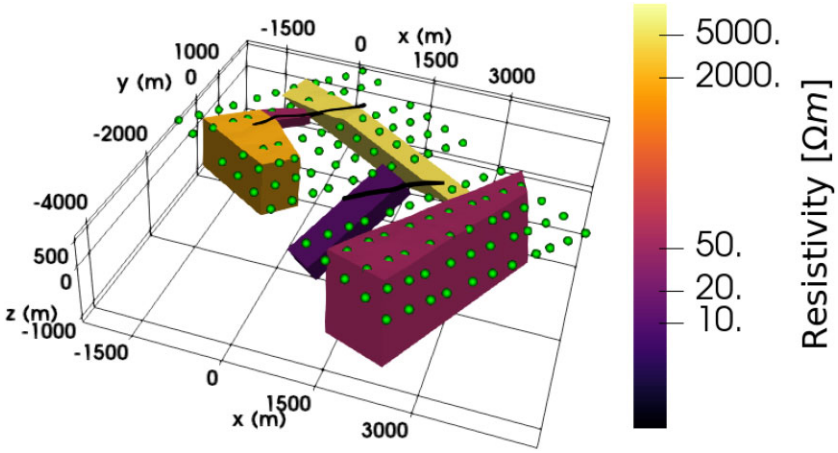


Figure 4. Example 2: Synthetic model showing the five anomalous structures in a 500 Ωm homogeneous background, the two transmitters (black) and all the receivers (green). Modified after Rochlitz *et al.* (2023).

itself was modelled after a real semi-airborne CSEM survey carried out near the city of Schleiz in Germany (Smirnova *et al.* 2019). As in Rochlitz *et al.* (2019), the topography was taken from a 25 m fine digital elevation model. However, instead of employing airborne receivers recording the magnetic field components, we place 11 lines with 11 receivers on the surface spanning an area of $4 \times 6\,\mathrm{km}^2$ and simulate the horizontal electric field components (E_x, E_y) and all components of the magnetic field. Note that the idea behind changing the survey setup is twofold: we can show the applicability to both the E- and B-field and we can compare the performance of the different forward operators using a land-based CSEM example which typically yields fewer data points than semi-airborne surveys (meaning smaller inversion size). A representation of the new synthetic model is given in Fig. 4, showing the anomalous bodies embedded in a 500 Ωm half-space, the transmitters and the receivers. It is further noted that the resistivity of the air space is set to 10^8 Ωm.

For the semi-airborne example, we adopt the 1071 airborne magnetic receivers presented in Rochlitz *et al.* (2023) which stem from 21 flight lines with 200 m spacing and 51 measuring points on each line separated by 120 m and are located 50 m above the digital elevation model. In addition, we keep the 11 ground receivers that measure the electric and magnetic fields. The resistivity model is the same as displayed in Fig. 4 and the placement of the airborne and surface receivers can be seen in fig. 1 in Rochlitz *et al.* (2023).

4 FORWARD MODELLING

For the purpose of the solver comparison, the MR3D model described in Section 3.1 is chosen as the test-bed for forward modelling with a focus on time and memory requirements, scaling behaviour and cross-platform applicability. As the code custEM was already

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

Table 1. Discretization information for the five meshes used.

Discretization information					
Model refinement	M1	M2	M3	M4	M5
# elements	400 790	2306 900	4470 474	6743 276	8748 576
# degrees of freedom	936 398	5372 032	10 399 316	15 677 176	20 350 580

validated against MR3D in Werthmüller *et al.* (2021) for the direct solver, a comparison between the responses of the direct and iterative solver is briefly shown in Appendix B1. Here, we present the performance analysis of the direct and iterative solvers for forward modelling using the MR3D model on five increasingly refined mesh discretizations as listed in Table 1. All simulations were performed on a Dell PowerEdge R940 server with four Intel Xeon Gold 6154 processors equipped with 18×(2) cores clocked at 3 GHz as well as with 48 LRDIMM 64 GB, DDR4-2666, Quad Ranks for a total of 3072 GB RAM.

All forward computations performed with the iterative framework are stopped when the relative residual of the outer GCR algorithm falls below 10^{-8}. The stopping tolerance for the inner pre-conditioned iterative solver is set to 10^{-2}. These tolerances are based on comparable iterative frameworks (see e.g. Grayver & Bürg 2014; Grayver & Kolev 2015; Qiu *et al.* 2021). It is further remarked that all simulations are run using first-order vector basic functions for the finite-element discretization. All the indicated solving times and memory requirements represent averages from simulations run on the six frequencies used for the MR3D model.

4.1 Computational complexities and scalability

First, we examine the time and memory complexities of both solution approaches. The measured time and memory demands can be found as numbers in the Appendix B2 in Table B1. In Fig. 5, we display the observed time and memory requirements of the solvers as a function of the number of degrees of freedom N_{dof} for simulations employing varying numbers of parallel MPI processes. To assess the orders of time and memory growth with increasing problem size, we compute the respective best fit according to the relationship $\left(\frac{n_1}{n_2}\right)^x = \frac{y_2}{y_1}$, where n_1 and n_2 are the number of degrees of freedom and y_1 and y_2 denote either the time or memory requirements. The exponent x then describes the approximate growth order. The obtained relations are shown as dotted lines and the corresponding growth orders are indicated in each plot.

For the direct solver, the solving time behaves nonlinearly and roughly proportional to the theoretical complexity of $\mathcal{O}(N_{dof}^2)$. The associated memory requirements approximately increase according to the theoretical order of $\mathcal{O}(N_{dof}^{4/3})$. The actual growth of time and memory demands range between $\mathcal{O}(N_{dof}^{1.88})$ and $\mathcal{O}(N_{dof}^{2.21})$, as well as $\mathcal{O}(N_{dof}^{1.37})$ and $\mathcal{O}(N_{dof}^{1.54})$, respectively. On the other hand, it can readily be seen that the iterative solver exhibits an approximately linear relation for both time and memory independent of the numbers of processes used.

Overall, the iterative solver is significantly less time and memory demanding than the direct solver and displays a negligible memory foot-print for growing number of processes. On the other hand, the direct solver is afflicted by a considerable memory overhead as the number of processes is increasing.

Next, we present a scalability study of the direct and iterative solver, that is, we assess how the solving time develops with successively increasing the number of processes. In Fig. 6, we display the obtained solving times in dependence of number of processes for both solvers. As before, we fit the observations using the aforementioned relation and plot them as dotted lines. The iterative solver scales almost linearly, whereas the direct solver displays a scaling proportional to approximately $\mathcal{O}(N_{MPI\,proc}^{-0.42})$. We can further observe that the achieved speedup for the iterative solver is clearly superior to the direct one. All in all, the iterative solver is characterized by excellent scalability whereas the direct solver scales considerably less favourable.

4.2 Behaviour for multiple right-hand side vectors

Hitherto, all experiments only considered a single source or right-hand side (RHS) vector, respectively. And as evidenced, the iterative solver requires less computational resources than the direct solver for single source problems for the tested problem sizes here. However and with inverse problems in mind, multiple source problems are of particular interest. We thus determined the times required to obtain solutions for 1, 10, 20, 30, 40 and 50 right-hand side vectors for the direct solver and the iterative algorithm with either an inner iterative or inner direct solver for three different problem sizes with 256 488, 1160 124 and 3607 252 degrees of freedom, respectively. To reflect the process count for inverse problems solving the forward problem and sensitivity matrix for multiple frequencies at the same time, we ran the multiple source problems using few processes, namely, four or twelve parallel processes. Fig. 7 depicts the obtained solving times in dependence of the number of RHS vectors. In addition, we linearly fit the observed solving times to extrapolate the approximate time requirements for larger numbers of RHS vectors as encountered in inverse problems.

From the indicated symbols and the corresponding fitted curves, one observes that the solution of additional RHS vectors results in considerably larger time requirements for the iterative solvers than for the direct solver. This is further mirrored in the slopes of the fitted curves. In fact, the direct solver starts outperforming the iterative algorithm in terms of time after one, three and nine additional RHS vectors for the three system sizes tested here. At large number of RHS vectors, the time requirement for the direct solver is reduced by about one order of magnitude in comparison to the iterative algorithm. The results in Fig. 7 also show that, depending on the problem size, it can be advantageous to employ an inner direct solver instead of the auxiliary-space pre-conditioner for the iterative algorithm. This can be seen looking at the observed time requirements and associated slopes for the two smaller problem sizes, where the observations for the algorithm using an inner direct solver yield lower simulation times than for an inner iterative solver after two and three RHS vectors for four MPI processes, and three and ten RHS vectors for twelve MPI processes. However, as more MPI processes are used and with increasing problem size, the auxiliary-space preconditioner proves to be slightly more time-efficient which is due to its better scaling and the lack of scalability of the direct solver. With inverse problems in mind, these observations are of particular relevance as they suggest which inner solution method one should

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

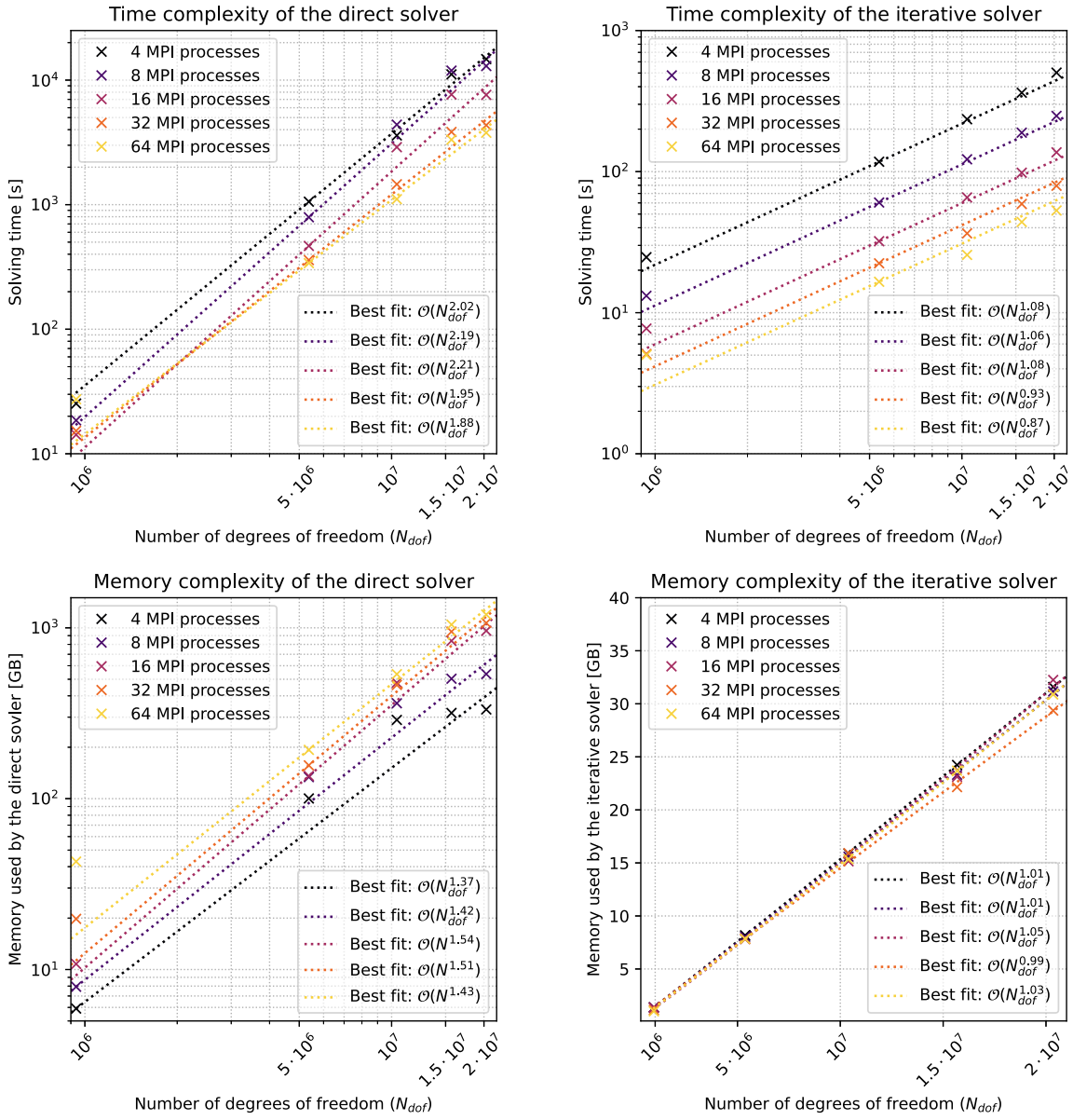


Figure 5. Time and memory complexity of the direct and iterative solver. For reference, best fits are indicated and plotted.

employ depending on the number of degrees of freedom and parallel processes, respectively.

4.3 Cross-platform applicability

Up to this point, we relied on a server with considerable computational power. In this part and on the basis of the aforementioned mesh discretizations of the MR3D model displayed in Table 1, we demonstrate that the iterative solver enables forward modelling on small computational platforms such as a workstation or a laptop.

The time and peak memory requirements for the MR3D models obtained with the iterative algorithm on a Intel(R) Core(TM) i7-7500U two-core processor @ 2.7GHz and with 32 GB RAM and Ubuntu 24.10 as operating system are plotted in Fig. 8. It is remarked that the equivalent quantities for the direct solution method are not plotted as the direct solver runs out of memory for the discretization M2 with 5372 032 degrees of freedom, whereas the iterative solver only runs out of memory for model discretization M5 with 20350 580 degrees of freedom. As on the cluster tests, it can be observed that both the solving time and the peak memory consumption scale approximately linear with increasing numbers of degrees of freedom. From these observations, one can confirm that the iterative solver facilitates forward modelling of large and complex models on small computing platforms with limited computational resources in a reasonable time.

5 INVERSE MODELLING

Here, we present the inverse modelling comparison of the direct and iterative solver for Example 2a and 2b (see descriptions in Section 3.2). As remarked, the focus lies on the time and memory required to obtain the explicit sensitivity matrix of various sizes. Nonetheless, an overall inversion comparison is shown as well.

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

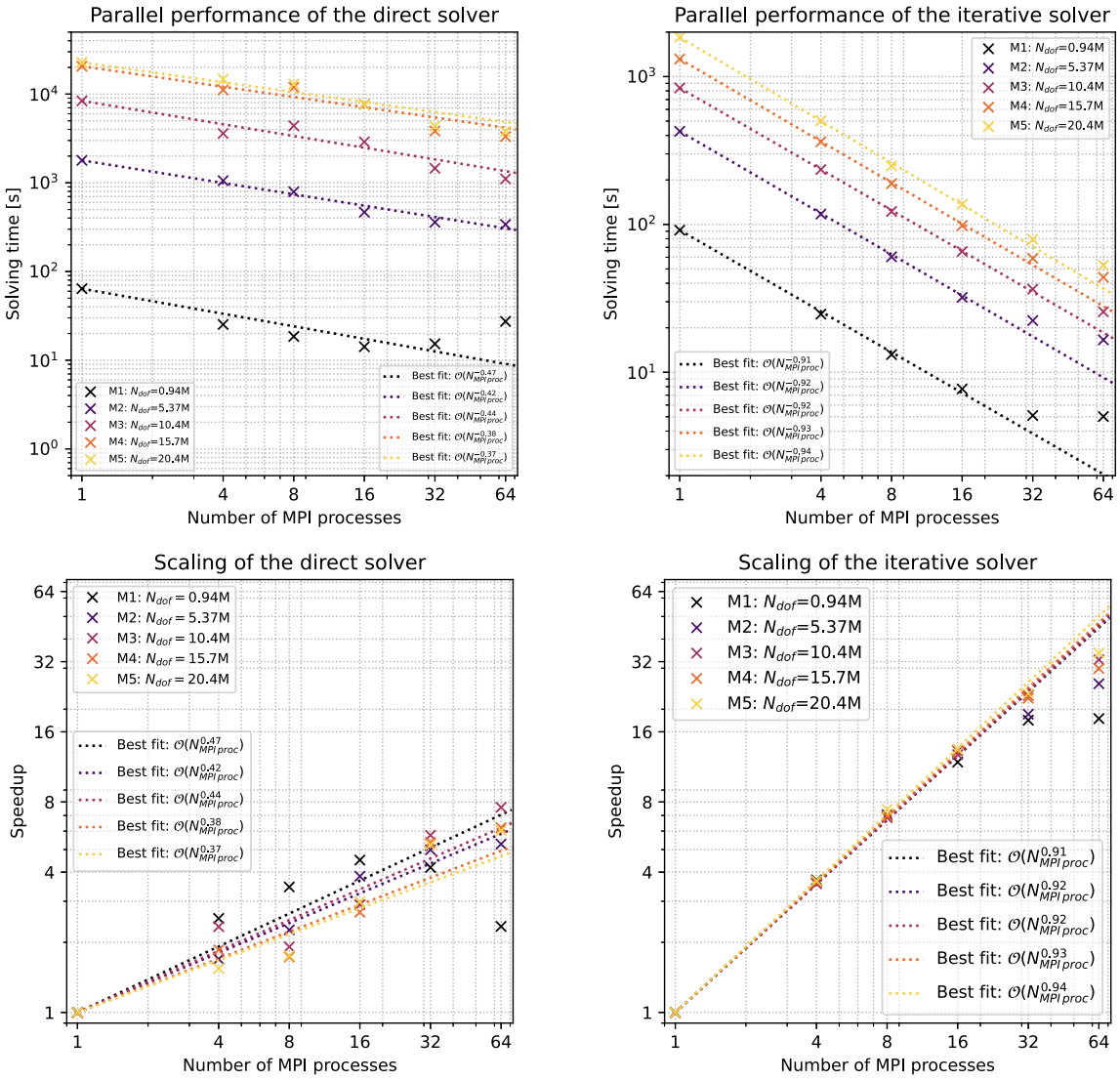


Figure 6. Performance analysis for the direct and iterative solver. For reference, best fits are indicated and plotted.

As for the forward modelling part, all simulations employ first-order basic functions for the finite-element discretization. Concerning the stopping criteria of the iterative framework, the forward responses are computed using the same tolerances as in the previous section, that is 10^{-8} and 10^{-2} for the outer and inner solver, respectively. As stated in Section 4, the choice for these tolerances is set in agreement with other comparable iterative frameworks (see e.g. Grayver & Bürg 2014; Grayver & Kolev 2015; Qiu *et al.* 2021). It is further remarked that the choice of the inner solver tolerance solely affects the solving time and outer iteration count of the whole iterative framework as demonstrated in Table 2 in Weiss *et al.* (2023) and can thus be set flexibly. In fact, the inner tolerance does not influence the solution accuracy of the outer solver. Moreover, the performance of the iterative framework increases with reducing inner solver tolerance as can be observed in table 2 in Weiss *et al.* (2023). Therefore, we opt to reduce the inner tolerance to 10^{-1} for inverse modelling compared to 10^{-2} for forward modelling in order to save computational resources.

5.1 Synthetic land-based CSEM model: inversion setup

Here, we briefly detail the computational setup used for the synthetic land-based CSEM model. We compute synthetic responses for 12 logarithmically spaced frequencies between 6 and 3500 Hz for the aforementioned model discussed in Section 3.2. For the subsequent study of the time requirement for the computation of the Jacobian and the inversion, we successively increase the number of receiver lines from one to eleven and all receivers obtain data from both transmitters yielding between 2640 and 26 146 data points N_d, where $N_d = N_{frequencies} \times N_{receivers} \times N_{components} \times N_{transmitters} \times 2$. All data are contaminated with five percent of Gaussian noise and data points are excluded if either the real or imaginary part lies below 0.5 pT A^{-1} for the magnetic fields.

The synthetic forward responses are computed using second-order polynomials on meshes with 327 327 to 352 308 elements, respectively. The responses for the inversion were computed using first-order vector basic functions for the finite-element discretization on meshes which do not include the anomalous structures, but restrict the element size in the region of interest and yield 248 983 to

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

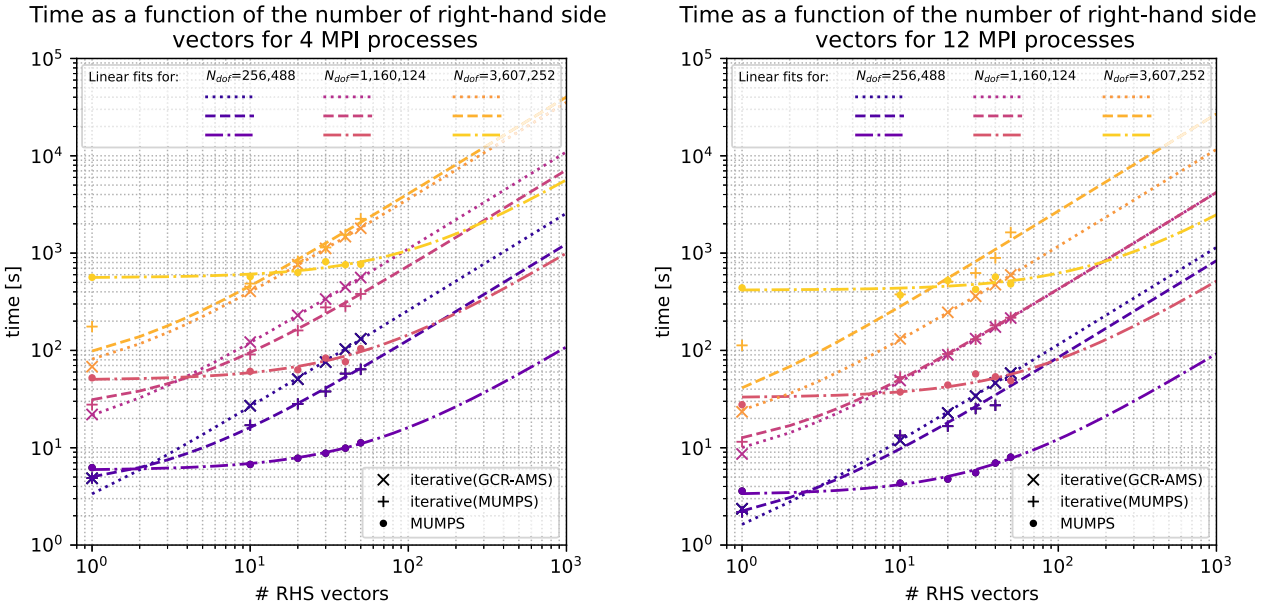


Figure 7. Solving time with increasing number of RHS vectors for three problem sizes for the direct solver (dots) and the iterative algorithm employing either an inner direct (pluses) or inner iterative solver (crosses).

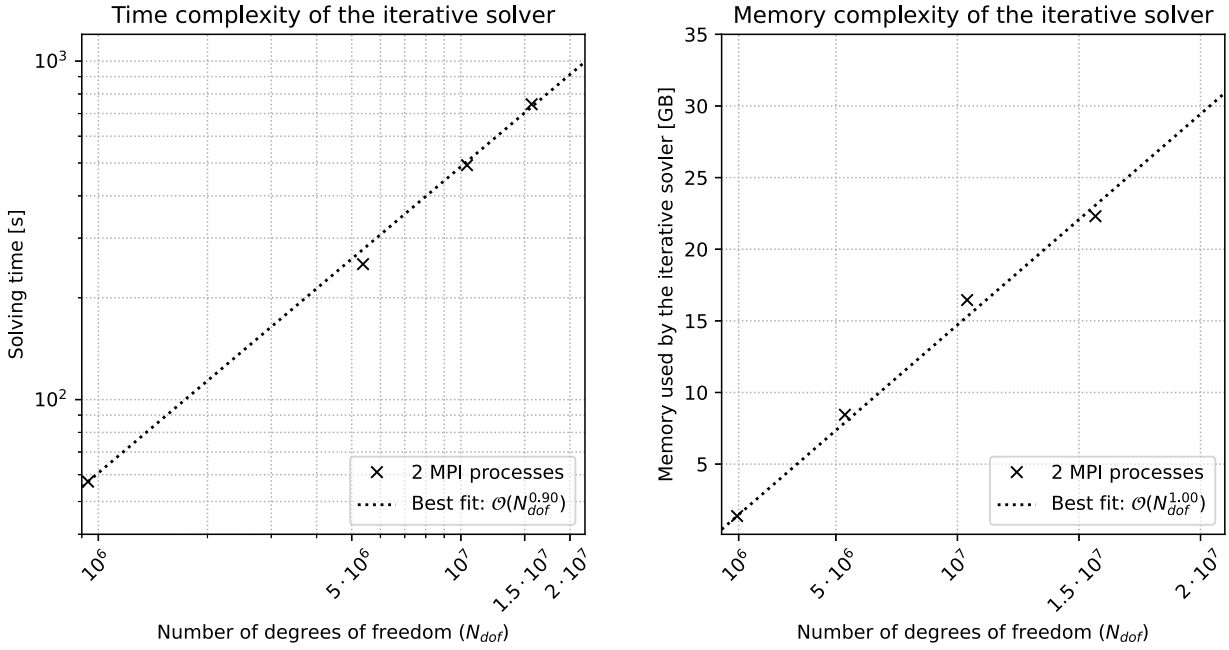


Figure 8. Time and memory complexity of the direct and iterative solver as obtained on a laptop. For reference, best fits are indicated and plotted.

473 055 elements and 123 502 to 235 230 inversion parameters, respectively. As the meshes for the forward and inverse modelling use vector basis functions of different order, the mesh quality control parameters of the meshing software TetGen (Si 2015) are adjusted accordingly and set to 1.3 and 1.6, respectively, and thus the inversion mesh differs naturally from the mesh used to compute the forward responses.

Any inversion performed henceforth is run with the direct solver MUMPS, the iterative algorithm using MUMPS as the inner solver and the iterative procedure with the auxiliary-space Maxwell solver as the inner solver. Each run uses 72 parallel processes, that is, six MPI processes per frequency and the regularization parameter λ is set to 10.

5.2 Synthetic land-based CSEM model: outer solver tolerances for the computation of the Jacobian

To gauge the influence of the outer solver tolerance on the accuracy of the sensitivity matrix and thus on the overall convergence of the inversion, we perform an analysis of the sensitivities generated by the direct as well as the iterative framework after the first inversion step for the example described in Section 3.2 and for eleven receiver

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

Table 2. Number of sensitivity values with magnitudes $\geq 10^{-6}$ for the electric and magnetic field components and the mean relative errors for the Jacobian computed with the iterative framework using either an inner iterative (top values) or inner direct solver (bottom values).

Number of sensitivities with magnitudes $\geq 10^{-6}$	E_x	E_y	H_x	H_y	H_z
$\Re$	1368 752	1375 805	752 985	748 987	746 073
$\Im$	891 245	890 682	471 788	471 109	462 996
		Outer solver tolerance			
Mean relative error (per cent)	10^{-3}	10^{-4}	10^{-6}	10^{-8}	10^{-12}
$\Re(E_x)$	7.8	2.6	4.8e^{-2}	3.5e^{-4}	4.9e^{-6}
	12.6	4.5	6.8e^{-3}	3.6^{-5}	4.9^{-6}
$\Im(E_x)$	11.5	3.8	5.2e^{-2}	4.2e^{-4}	3.4e^{-5}
	22.0	7.6	1.2e^{-2}	9.2e^{-5}	3.4e^{-5}
$\Re(E_y)$	7.8	2.6	4.8e^{-2}	3.4e^{-4}	4.9e^{-6}
	12.5	4.4	6.8e^{-3}	3.6e^{-4}	4.9e^{-6}
$\Im(E_y)$	11.4	3.8	5.2e^{-2}	4.1e^{-4}	3.5e^{-5}
	22.0	7.6	1.2e^{-2}	9.2e^{-5}	3.5e^{-5}
$\Re(H_x)$	31.5	11.7	9.5e^{-2}	9.9e^{-4}	5.6e^{-4}
	30.6	16.5	6.0e^{-2}	8.8e^{-4}	5.4e^{-4}
$\Im(H_x)$	30.9	11.5	1.2e^{-1}	2.6e^{-3}	2.1e^{-3}
	34.5	18.8	5.8e^{-2}	2.3e^{-3}	2.0e^{-3}
$\Re(H_y)$	31.6	11.8	9.6e^{-2}	1.0e^{-5}	5.6^{-4}
	30.9	16.7	6.1e^{-2}	8.9e^{-4}	5.5^{-4}
$\Im(H_y)$	31.1	11.7	1.3e^{-1}	2.6e^{-3}	2.1e^{-3}
	34.8	19.0	5.9e^{-2}	2.3e^{-3}	2.0e^{-3}
$\Re(H_z)$	31.4	11.5	9.1e^{-2}	9.7e^{-4}	5.5^{-4}
	30.4	16.3	5.8e^{-2}	8.7e^{-4}	5.4e^{-4}
$\Im(H_z)$	30.8	11.4	1.2e^{-1}	2.6e^{-3}	2.1e^{-3}
	34.2	18.6	5.7e^{-2}	2.3e^{-3}	2.0e^{-3}

lines and computational settings as described in Section 5.1. In particular, we computed the Jacobian with the direct solver MUMPS, the iterative algorithm using MUMPS as the inner solver and the iterative procedure with the auxiliary-space Maxwell solver as the inner solver. We test outer solver tolerances of 10^{-3}, 10^{-4}, 10^{-6}, 10^{-8} and 10^{-12}.

Given the considerable size of the Jacobian of $26\,146 \times 235\,230$, the comparison is performed for the frequency of 200 Hz and compromises $2 \times 121 \times 5 \times 235,230 \times 2$ real-valued sensitivities. They relate how perturbations of the model parameters affect the data, and a small sensitivity entry thus indicates that an observation is insensitive to the considered model parameter, meaning that large variations of that model parameter would only result in a small change in the data. Consequently, the highest sensitivity values predominately determine the direction of the inversion. The sensitivities generally span multiple orders of magnitude ranging from values around 10^{-2} to 10^{-15}. We therefore restrict the analysis to sensitivity entries with magnitudes larger than 10^{-6} as smaller sensitivity values have extremely limited influence on the overall course of the inversion. We filter out these sensitivities for the Jacobian obtained with the direct solver yielding a total of 8180 422 sensitivity values as tabulated by components in Table 2. We then retrieve the corresponding sensitivities for the Jacobians computed with the iterative approaches for varying outer solver tolerances of 10^{-3}, 10^{-4}, 10^{-6}, 10^{-8} and 10^{-12} and a respective inner solver tolerance of 10^{-1}.

To assess whether the tested outer solver tolerances suffice to compute adequately accurate approximations of the sensitivity matrix, the mean relative errors between the Jacobian generated by the direct solver and the approaches of the iterative framework are computed. In addition, the error-weighted data misfits (χ^2) are assessed after one iteration step. The mean relative errors for the real and imaginary parts of all electric and magnetic field components are given in Table 2. As the relative errors are almost identical for the real and imaginary parts of both fields, Fig. 9 represents the error convergence with respect to outer solver tolerance for the real and imaginary parts of the electric and magnetic fields. Table 3 shows the error-weighted misfits after one iteration step. These tables and the figure demonstrate that outer solver tolerances $\leq 10^{-6}$ yield highly accurate sensitivities. Examining the assessing measures more closely, one can see that the mean relative errors for outer solver tolerances of 10^{-4} and 10^{-3} decrease for sensitivities with magnitudes $\geq 10^{-5}$ compared to magnitudes $\geq 10^{-6}$. In particular, the mean relative errors for the electric and magnetic field components drop from 10 per cent and 31 per cent to 5 per cent and 11 per cent for the fully iterative-based framework using an outer solver tolerance of 10^{-3}. A similar decrease is seen for the mean relative errors for both field components when using the iterative framework with an inner direct solver and an outer solver tolerance of 10^{-3}, namely drops from 17 per cent and 33 per cent to 5 per cent and 11 per cent for the electric and magnetic components, respectively. This also applies to the mean relative errors obtained with the iterative solver using an outer solver tolerance of 10^{-4} and inner iterative solver where the errors fall from 3 per cent and 12 per cent to 1 per cent and 4 per cent for the electric and magnetic field components. For the iterative framework using an inner direct solver, the errors for the electric and magnetic field components shrink from 6 per cent and 18 per cent to 2 per cent and 8 per cent, respectively. Recalling that the largest sensitivities predominately exercise the direction of the inversion, these reductions indicate that an outer solver tolerance of 10^{-4} generates sufficiently accurate sensitivity matrices and that even an outer solver tolerance of 10^{-3} might yield acceptable approximations of the Jacobian. This observation is further substantiated when looking at the error-weighted data misfits in Table 3 which show that the data misfit achieved with an outer

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

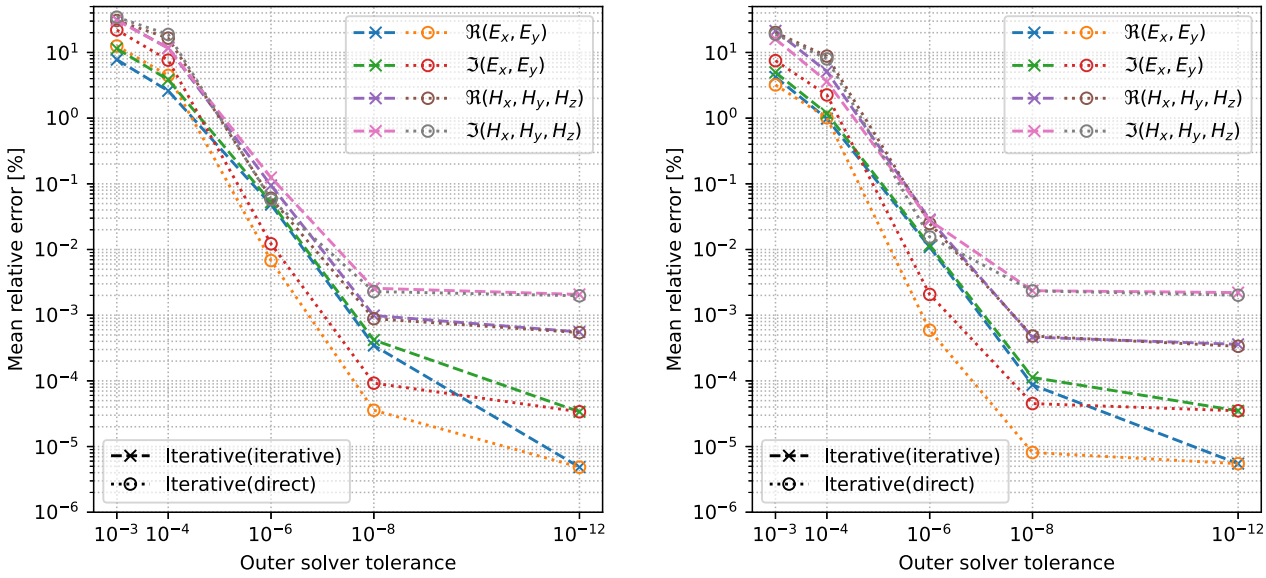


Figure 9. Mean relative errors for the sensitivities computed with the iterative framework using varying outer solver tolerances for all electric and magnetic field components. The left panel shows the errors for sensitivity values with magnitudes $\geq 10^{-6}$ and the right panel gives the corresponding errors for entries with magnitudes $\geq 10^{-5}$.

Table 3. The error-weighted misfit χ^2 for the iterative framework using either an inner iterative or inner direct solver.

	χ^2				
Direct solver	99.55				
	Outer solver tolerance				
	10^{-3}	10^{-4}	10^{-6}	10^{-8}	10^{-12}
Inner iterative solver	104.33	99.13	99.59	99.72	99.47
Inner direct solver	105.29	97.51	100.19	99.57	99.61

solver tolerance of 10^{-4} is in good agreement with the data misfit obtained when using a direct solver ascertaining that the inversions step in the same direction. Even for an outer solver tolerance of 10^{-3}, the error-weighted data misfits are fairly close ensuring that the inversions step approximately in the right direction.

Given the observed mean relative errors together with the error-weighted data misfits, we select an outer solver tolerance of 10^{-4} as the default outer solver tolerance to obtain the sensitivity matrix in inverse modelling for the coming inversion examples.

5.3 Synthetic land-based CSEM model: inversions

As described in Section 5.1, we study the time requirement for the computation of the Jacobian and the inversion for six configurations starting from only one receiver line adding two lines subsequently until reaching eleven receiver lines. For each case, we ran the inversion with the direct solver MUMPS, the iterative algorithm using MUMPS as the inner solver and the iterative procedure with the auxiliary-space Maxwell solver as the inner solver, thus totalling 18 inversion runs. Inversions performed with the iterative solver used outer and inner stopping tolerances of 10^{-4} and 10^{-1}, respectively.

Before discussing various performance parameters of the different solving approaches, we first show the recovered resistivity models for the model with eleven receiver lines (see Fig. 4) in Fig. 10. We mask resistivity values between 200 and 1000 Ωm to better show the recovery of the resistivity anomalies. All three solution approaches recover the anomalous bodies well. To assess the differences between the models, we employ the normalized root-mean square difference (NRMSD) between the logarithms of cell resistivity values of two different models ρ_{m_1} and ρ_{m_2} expressed by

$$\text{NRMSD} = \frac{|(\rho_{m_1}) - (\rho_{m_2})|}{(|\rho_{m_1}| + |\rho_{m_2}|)/2} \times 100 \; \textit{per cent}. \tag{4}$$

In Fig. 11, we plot the NRMSD between the model obtained with the direct approach and each iterative approach. The following observations stand out: (1) The fully iterative approach yields lower differences than the iterative approach employing an inner direct solver which is reasonable considering the final misfits. In particular, the differences are considerably larger for the largest conductor and the one at depth for the iterative-direct approach; (2) The resistive anomalies yield the smallest discrepancies around one percent; (3) The largest differences coincide with source and receiver locations on the surface.

Next, we look at performance parameters displayed in Table 4. We list the number of data points and inversion parameters and we show the respective iteration count to reach convergence, the corresponding error-weighted data misfit χ^2, the required inversion time, the average time to construct the sensitivity matrix and the peak memory consumption for either the construction of the sensitivity matrix or the computation of the forward responses, respectively, and for the inversion overall.

Overall, one can observe that all the inversions converge to data misfits χ^2 of similar values in comparable numbers of steps. After an initial drop of the misfits upon introducing additional data points, the misfits level out and remain almost constant. In terms of overall inversion time, we see that inversions relying on a direct solution approach require the least time to converge, whereas the iterative approach in combination with a direct inner solver comes second and the fully iterative solver is last. The differences in time between the three approaches mainly stem from the average times required to build the Jacobian as well as additional iteration steps required to

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

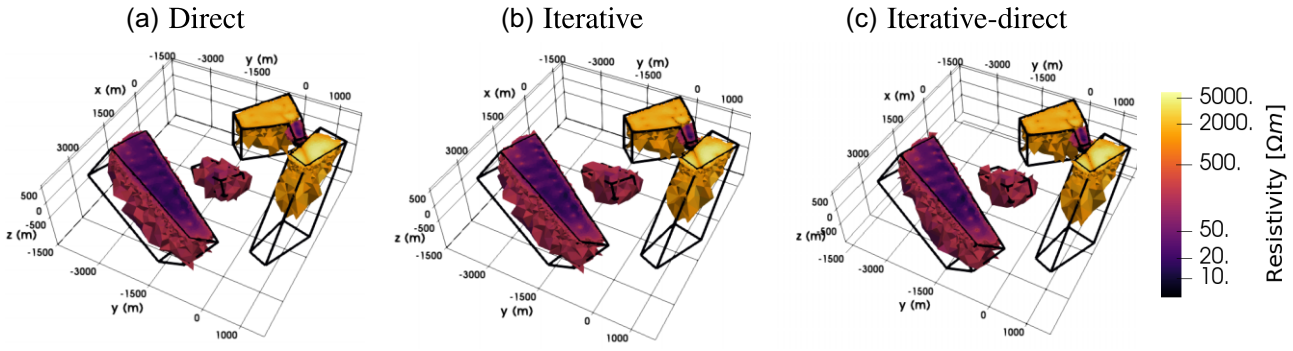


Figure 10. Recovered resistivity models for the three solution approaches. For comparison, the outlines of the structures are shown as well.

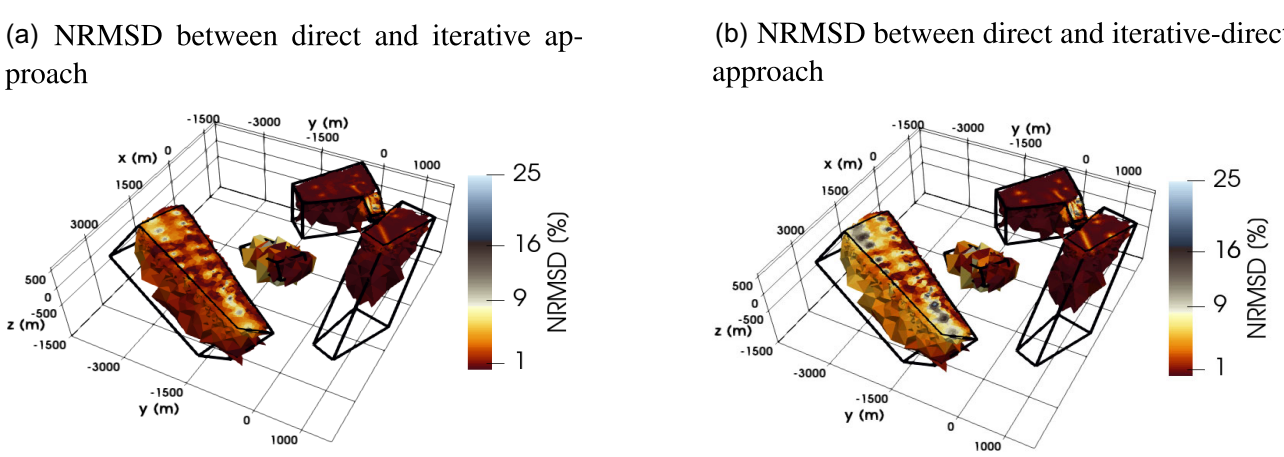


Figure 11. Normalized root-mean square differences (NRMSD) between results from the direct approach and from the iterative algorithm using either an inner pre-conditioned iterative solver (left panel) or an inner direct solver (right panel).

Table 4. Characteristic quantities used to asses the inversions. Note that quantities with three values represent the simulations with the direct solver (top value), the iterative algorithm using an inner direct solver (middle value) and the iterative solver with the AMS pre-conditioner (bottom value).

	l_1	l_3	l_5	l_7	l_9	l_{11}
Data points	2440	7298	12 122	16 804	21 550	26 146
Inversion parameters	123 502	142 651	163 669	185 365	204 088	235 230
Numbers of iterations	6	7	10	9	9	10
	6	7	10	12	11	8
	6	7	7	10	11	8
Error-weighted data misfit χ^2	1.86	1.42	1.45	1.4	1.44	1.37
	1.93	1.44	1.47	1.38	1.44	1.49
	1.88	1.43	1.65	1.42	1.42	1.42
Inversion time (hr)	1.60	3.46	7.54	10.9	17.2	23.3
	1.67	3.72	8.38	15.0	21.9	21.8
	1.75	4.28	6.34	14.4	23.2	23.9
Average time to obtain Jacobian (min)	11.1	12.4	17.3	29.3	38.0	40.2
	11.8	17.3	22.4	29.8	49.1	46.6
	12.9	22.5	26.4	40.7	59.7	64.8
Memory consumption for Jacobian (GB)	86.5	100.1	116.1	137.8	140.9	187.2
	58.5	84.5	97.9	115.1	134.2	158.8
	43.7	56.9	69.4	79.4	96.0	118.9
Memory consumption overall (GB)	86.5	100.1	116.1	137.8	193.2	274.8
	58.5	84.5	97.9	136.8	181.9	268.3
	43.7	56.9	89.5	137.5	182.2	274.8

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

Table 5. Characteristic quantities for three outer solver tolerances when using the iterative algorithm as forward solver. Note that the inner solver can either be a direct solver, namely MUMPS (upper left) or and iterative solver pre-conditioned with auxiliary-space Maxwell solver AMS (lower right).

Outer solver tolerance	10^{-2}	10^{-3}	10^{-4}	10^{-5}
Inner solver	MUMPS / AMS	MUMPS / AMS	MUMPS / AMS	MUMPS / AMS
Number of iterations	8 / 8	13 / 11	8 / 8	11 / 10
Error-weighted data misfit χ^2	1.70 / 1.69	1.40 / 1.36	1.49 / 1.42	1.33 / 1.35
Inversion time [h]	20.2 / 20.6	30.3 / 29.0	21.8 / 23.9	29.1 / 29.5
Average time to obtain Jacobian [min]	40.0 / 39.6	42.0 / 60.1	46.6 / 64.8	58.2 / 79.3

converge. Nonetheless and given the duration of the inversion runs, the iterative approaches do not fall far behind the direct solution method. Lastly, the memory consumption of each inversion was monitored and the observed peak memory requirements are either bound by the forward and sensitivity computations or by inversion tasks as seen in Table 4, respectively. It turns out that the forward and sensitivity calculations represent the bottleneck for the smaller data sets, whereas inversion-related tasks become more memory-demanding than the forward and sensitivity computations for data sets of a certain size (cf. memory consumption of the Jacobian and overall for models l_7 and l_9). It is remarked that the memory requirement for the forward problem and the Jacobian also grows with increasing data set size and that it might become the memory bottleneck for very large problems.

5.4 Synthetic land-based CSEM model: the influence of the outer solver tolerance on inversions

Thus far, the iterative approaches used the default outer solver tolerance of 10^{-4}. We recall that even an outer solver tolerance of 10^{-3} might be sufficient to guide the inversion as alluded to above. Here, we investigate the influence of the outer solver tolerance across inversions based on the convergence history, required number of iterations and associated time requirements. We therefore also ran inversions of model l_{11} for outer solver tolerances of 10^{-2}, 10^{-3} and 10^{-5}. The results are displayed in Table 5. The least restrictive tolerance of 10^{-2} yields a larger error-weighted data misfit than the more restrictive tolerances. The lowest data misfits are obtained with the most restrictive tolerance of 10^{-5}. Interestingly, the inversions carried out with a tolerance of 10^{-3} boast lower data misfits than for a tolerance of 10^{-4}. The number of iteration steps varies between 8 and 13 and one may correlate the number of steps with reduced overall data misfit. Increasing the outer tolerance of the iterative solver results in longer computation times for the sensitivity matrix. As the calculation of the Jacobian takes considerably less time than performing one inversion step, the overall time requirements are correlated with the number of inversion steps.

For comparison, we plot the NRMSD of inversions using the iterative solver with an inner AMS-pre-conditioned GCR solver for outer solver tolerances of 10^{-3} to 10^{-5} in Fig. 12. Note that these simulations were chosen as they obtained the lowest data misfits. We observe decreased differences for the largest as well as for the buried conductors for inversions with tolerances 10^{-3} and 10^{-5} when compared to the inversion with tolerance 10^{-4}. The inversion with tolerance 10^{-3} is most influenced by the source and receiver imprint at the surface. The most restrictive tolerance yields the lowest discrepancies.

5.5 Semi-airborne CSEM model: extrapolation to a large scale survey

As for the land-based survey, we simulate synthetic responses for 12 logarithmically spaced frequencies between 6 and 3500 Hz for the semi-airborne model discussed in Section 3.2. With this example, we investigate how the iterative approaches deal with a larger data set consisting of 133 648 data points, which is about one order of magnitude more than for the land-based survey, and how the performance compares to the direct approach.

The synthetic forward responses were computed using second-order polynomials on a mesh with 417 021 elements. The responses for the inversion were computed using first-order vector basic functions on a mesh that excludes the anomalous structures and restricts the element size in the region of interest yielding 125 488 inversion parameters. Due to the use of different polynomial orders, the forward and inverse meshes naturally avoid the inverse crime.

As we are interested in the computational performance of the various solution approaches, we ran three inversions for only three iteration steps, namely one with the direct solver MUMPS, one with the iterative algorithm using MUMPS as the inner solver and one with the iterative procedure with the auxiliary-space Maxwell solver as the inner solver.It is further remarked that the inversions performed with the iterative solver use outer and inner stopping tolerances of 10^{-3} and 10^{-1}, respectively. The outer solver tolerance is reduced due to the finding in Section 5.4, which showed that an outer solver tolerance of 10^{-3} yields an accurate inverse model.

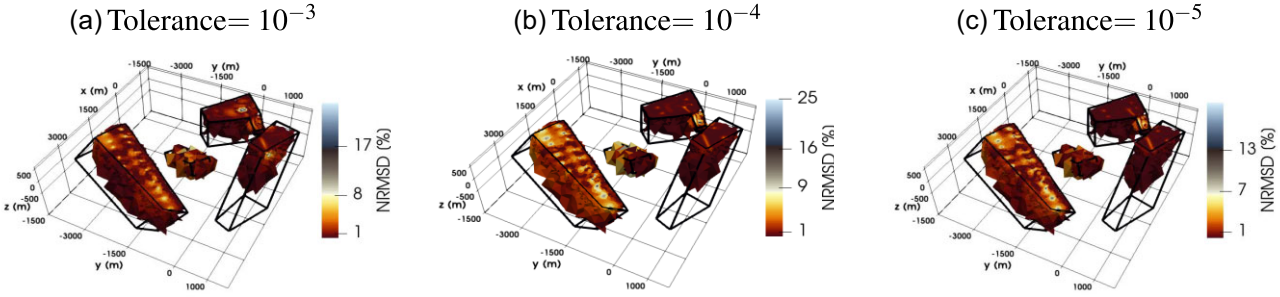


Figure 12. Normalized root-mean square differences (NRMSD) between results from the direct approach and from the iterative algorithm using an inner pre-conditioned iterative solver for three different outer solver tolerances.

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

Table 6 displays the performance parameters for the semi-airborne survey run for three iterations. As for the land-based example, the direct approach performs best in terms of overall time requirements as well as average time needed to compute the data sensitivities. In terms of memory demands, the direct approach requires most memory for the computation of the Jacobian. The fully iterative approach requires about 50 per cent more total time than the direct approach while saving approximately 40 per cent memory for the computation of the sensitivity matrix. The mixed iterative-direct approach only takes slightly more time than the direct approach (≈ 11 *per cent*), but only reduces the memory consumption for the Jacobian computation by about the same percentage.

6 DISCUSSION

6.1 Forward modelling

The presented forward modelling study showed that the implemented iterative solver is considerably less resource-intensive than the direct solution approach. A scalability study attests to the excellent scaling behaviour of the iterative solver. It can thus be concluded that stand-alone forward modelling tasks are best carried out using an iterative solver with suited pre-conditioning. It can further be remarked that the iterative solver is extremely memory-friendly, which enables computations of complex and large models on computing platforms with limited resources in very reasonable times. This is particularly significant as it opens up forward modelling without access to high-performance and high-resource platforms and facilitates cross-platform applicability.

On the other hand, for problems requiring solutions for multiple right-hand side vectors, the direct solver clearly outperforms the iterative procedure in terms of computational time as the expensive matrix factorization can be reused at low cost. Our investigation further shows that the iterative algorithm employing a direct inner solver may be more time efficient than an inner iterative solver with the auxiliary-space pre-conditioner for multiple right-hand side vectors depending on the problem size and number of parallel processes.

Viewed from the context of forward modelling, the iterative approaches may be used for multiple-source problems even though it adds about one order of magnitude in time requirement compared to the direct solver. With the inverse problem in mind, this favours the use of a direct solution methods but also indicates that iterative approaches are not prohibitively expensive and may offset the time requirement by the lower memory consumption.

We recall that all simulations employ first-order vector basic functions for the finite-element discretization. This is due to the underlying software FEniCS which does not provide the computation of the discrete gradient operators that map derivatives of finite element functions into other finite element spaces for higher order basis functions. This discrete gradient matrix is required when constructing the auxiliary-space Maxwell pre-conditioner. Nonetheless, we remark that computations based on higher order basis functions could further improve modelling accuracy while reducing the problem size. It is noted that the auxiliary-space Maxwell pre-conditioner supports higher order approximations and that the presented iterative algorithm is robust with respect to higher order basis functions (see Grayver & Kolev 2015).

6.2 Inverse modelling

The inverse modelling examples indicate that iterative algorithms may constitute an alternative to direct solvers in the framework of inversion. The land-based CSEM scenario showed that the time requirements for inversions based on explicit sensitivity calculations are of the same order for both the direct and iterative forward operators. In addition, the use of iterative-based algorithms resulted in lower memory demands for the computation of the full sensitivity matrix. However, it is noted that the peak memory consumption is not caused by the computation of the Jacobian, but tasks related to solve the inverse system using the framework pyGIMLi. Hence, the iterative solvers cannot reduce overall memory consumption.

These examples further indicate that small scale inversions, that is to say inversions with approximately 10 000 data points and 140 000–150 000 inversion parameters, ought to be feasible on a laptop or workstation having access to more than 64 GB RAM when using the fully iterative approach (cf. Table 4). It is remarked that the simulation time would likely increase as one could not compute the sensitivities for all frequencies at once. It can further be noted that performing inversions with less frequencies would allow to increase the number of data points and/or inversion parameters.

The semi-airborne example indicates the applicability of the mixed-iterative forward operator in case of large-scale inversions of semi-airborne or marine surveys. To put the survey size of the semi-airborne example in context, we note that the storage for the full sensitivity matrix for this examples is equal to around 134 GB which recalling Fig. 1 constitutes an inverse problem of considerable size. In light of this, the performance of the mixed iterative-direct approach looks promising for this approach considering that the additional time is small compared to the direct-based forward engine.

Various tolerances were tested for the outer iterative algorithm and the following remarks are made: More restrictive tolerances result in higher computational costs (time) as more steps are required of the iterative solver to compute each entry of the sensitivity matrix. On the other hand, entries calculated with increasingly restrictive tolerances approach the Jacobian entries computed by a direct solver (cf. Table 2 and Fig. 9). It is remarked that an outer solver tolerance of 10^{-6} generates highly accurate sensitivities. Hence, the recovered resistivity models between the direct and iterative forward operators are in better agreement with each other and the influence of the surface receivers and transmitters is reduced.

From our experience, a tolerance of 10^{-4} for the outer solver best balances the need for a fast and sufficiently accurate computation of the Jacobian. Varying outer solver tolerances for each inversion step were tested and dismissed as it did not yield any advantages over having a fixed tolerance.

It is recalled that the Jacobian is computed explicitly which has its benefits and drawbacks. Other inversion frameworks (cf. Egbert 2012; Egbert & Kelbert 2012; Grayver 2015) avoid computing the Jacobian explicitly by recasting the problem in order to compute one row or one column of the Jacobian by solving the governing equation once. Hence, evaluating a matrix-vector product with the Jacobian or its transpose yielding the Jacobian entries for all receivers for one source and frequency is equivalent to one forward solve. This might prove to suit an iterative forward operator better if the number of required forward solves can be reduced significantly compared to the number of forward computations needed for the full Jacobian. The numbers of forward solves for both approaches is well known and of orders $\mathcal{O}(2N_fN_rN_tN_c)$ and $\mathcal{O}(2N_{it}N_fN_t)$ for the full Jacobian and strategies evaluating matrix-vector products with the Jacobian (Grayver *et al.* 2013), where it is emphasized that N_{it} corresponds to the number of iterations required to compute a sufficiently accurate model update and not to the number of inversion steps.

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

Table 6. Characteristic quantities used to asses the inversions.

	Direct	Fully iterative	Mixed iterative-direct
Error-weighted data misfit χ^2	6.94	6.42	6.45
Inversion time (hr)	29.6	43.7	33.7
Average time to obtain Jacobian (hr)	3	8.1	3.9
Memory consumption for Jacobian (GB]	339.8	239.8	302.4
Memory consumption overall (GB)	793.2	791.5	792.6

Alternatively, one could solve the inverse problem by minimizing the penalty function directly using a gradient-based optimization method such as quasi-Newton [e.g. the limited-memory BFGS algorithm (Liu & Nocedal 1989)] or nonlinear conjugate-gradient algorithms. Both approaches avoid the explicit computation of the Jacobian and are considered fast and memory-efficient, but less capable meaning that they potentially require many more iterations to minimize the objective function, which may not be guaranteed.

Lastly, we recall that the iterative framework is based on matrix properties and that the solver is independent of the underlying mesh discretization as well as the distribution of the physical properties and the frequencies employed. Hence, the iterative framework does not depend on the particular model, and the conclusions are expected to be transferable to other models and even to modelling problems in magnetotellurics.

7 CONCLUSIONS AND OUTLOOK

We evaluated the computational performance of an iterative framework for both forward and inverse modelling in CSEM applications. We conclude that forward modelling problems are best solved using the presented iterative framework and that the framework allows simulations of large and complex problems even on small computing platforms such as laptops in very reasonable time.

For inverse problems, the iterative framework proves promising as indicated by the performance comparison. The tested iterative forward operators perform equally well in terms of time as the direct one for small to intermediate scale inversions while reducing the memory demands for the computations of the forward responses and the data sensitivities. For large scale inversions, the mixed iterative-direct-based forward solver constitutes a viable alternative to the direct operator as it is only slightly less time-efficient, but slightly more memory-efficient.

Future work include the extension to higher order vector basis functions for the finite-element discretization using FEniCSx instead of FEniCS for the iterative framework. Further, the implementation of a memory-saving inversion framework that minimizes the gradient of the objective function directly using a corresponding optimization technique is intended as the reduction of required forward solves to compute the sensitivity matrix might benefit iterative-based forward operators by improving the associated computational performance. This could make inversion of EM data sets from airborne, marine and other large-scale surveys possible on common computing platforms.

ACKNOWLEDGMENTS

We thank Colin Farquharson and an anonymous reviewer for their valuable comments and suggestions that helped improving this work. We sincerely thank the developers of FEniCS, TetGen, pyGIMLi, PETSc, hypre and all other underlying libraries for their effort on providing and maintaining open-source software tools over the years. The work was funded as part of the DESMEX II and DESMEX-REAL projects by the Germany Ministry for Education and Research (BMBF) in the framework of the research and development program Fona-r4 under grant 033R130DN and 033R385D, respectively.

DATA AVAILABILITY

The custEM toolbox is open-source and available under the GNU Lesser General Public License (LGPL). The complete code is documented utilizing the Python Application Programmer's Interface (API). The documentation is available on ReadtheDocs at https://custem.readthedocs.io/en/latest/ including instructions, notes and tutorials.

The data for the Marlim model is available at Zenodo and the modelling scripts can be found at https://gitlab.com/custem/custem-examples/-/tree/main/IterativeSolver/Marlim.

The scripts for the inversion examples are available at https://gitlab.com/custem/custem-examples/-/tree/main/IterativeSolver/Inversion and our work is thus reproducible.

All python scripts are compatible with custEM version 1.6 (May 2025) and pygimli 1.5.

REFERENCES

Alnaes, M.S. *et al.*, 2015. The FEniCS project version 1.5, *Arch. Numer. Softw.,* **3**. doi: 10.11588/ans.2015.100.20553.

Amestoy, P.R., Duff, I.S. & L'Excellent, J.-Y., 2000. Multifrontal parallel distributed symmetric and unsymmetric solvers, *Comp. Methods Appl. Mech. Eng.,* **184**(2-4), 501–520.

Avdeev, D.B., 2005. Three-dimensional electromagnetic modelling and inversion from theory to application, *Surv. Geophys.,* **26**(6), 767–799.

Axelsson, O. & Vassilevski, P.S., 1991. A black box generalized conjugate gradient solver with inner iterations and variable-step preconditioning, *SIAM J. Matrix Anal. Appl.,* **12**(4), 625–644.

Axelsson, O., Neytcheva, M. & Ahmad, B., 2014. A comparison of iterative methods to solve complex valued linear algebraic systems, *Numer. Algorithms,* **66**(4), 811–841.

Axelsson, O., Farouq, S. & Neytcheva, M., 2016. Comparison of preconditioned Krylov subspace iteration methods for PDE-constrained optimization problems, *Numer. Algorithms,* **73**(3), 631–663.

Axelsson, O., Farouq, S. & Neytcheva, M., 2017. A preconditioner for optimal control problems, constrained by Stokes equation with a time-harmonic control, *J. Comput. Appl. Math.,* **310,** 5–18.

Balay, S. *et al.*, 2024. PETSc/TAO users manual, Tech. Rep. ANL-21/39 - Revision 3.21, Argonne National Laboratory, U.S.A..

Becken, M. *et al.*, 2020. DESMEX: A novel system development for semi-airborne electromagnetic exploration, *Geophysics,* **85**(6), E253–E267.

Börner, R.-U., 2010. Numerical modelling in geo-electromagnetics: advances and challenges, *Surv. Geophys.,* **31**(2), 225–245.

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

Carvalho, B.R. & Menezes, P. T.L., 2017. Marlim R3D: a realistic model for CSEM simulations-phase I: model building, *Braz. J. Geol.,* **47,** 633–644.
Castillo-Reyes, O., de la Puente, J. & Cela, J.M., 2018. PETGEM: A parallel code for 3D CSEM forward modeling using edge finite elements, *Comput. Geosci.,* **119,** 123–136.
Castillo-Reyes, O., de la Puente, J., García-Castillo, L.E. & Cela, J.M., 2019. Parallel 3-D marine controlled-source electromagnetic modelling using high-order tetrahedral Nédélec elements, *Geophys. J. Int.,* **219**(1), 39–65.
Castillo-Reyes, O., Rulff, P., Schankee Um, E. & Amor-Martin, A., 2023. Meshing strategies for 3d geo-electromagnetic modeling in the presence of metallic infrastructure, *Comput. Geosci.,* **27**(6), 1023–1039.
Chave, A.D., 2017. Estimation of the magnetotelluric response function: the path from robust estimation to a stable maximum likelihood estimator, *Surv. Geophys.,* **38**(5), 837–867.
Chave, A.D. & Jones, A.G., 2012. *The Magnetotelluric Method: Theory and Practice,* Cambridge Univ. Press.
Cockett, R., Kang, S., Heagy, L.J., Pidlisecky, A. & Oldenburg, D.W., 2015. SimPEG: An open source framework for simulation and gradient based parameter estimation in geophysical applications, *Comput. Geosci.,* **85,** 142–154.
Commer, M. & Newman, G.A., 2008. New advances in three-dimensional controlled-source electromagnetic inversion, *Geophys. J. Int.,* **172**(2), 513–535.
Correa, J.L. & Menezes, P.T., 2019. Marlim R3D: A realistic model for controlled-source electromagnetic simulations—Phase 2: The controlled-source electromagnetic data set, *Geophysics,* **84**(5), E293–E299.
da Silva, N.V., Morgan, J.V., MacGregor, L. & Warner, M., 2012. A finite element multifrontal method for 3D CSEM modeling in the frequency domain, *Geophysics,* **77**(2), E101–E115.
Dalcin, L.D., Paz, R.R., Kler, P.A. & Cosimo, A., 2011. Parallel distributed computing using Python, *Adv. Water Resour.,* **34**(9), 1124–1139.
Egbert, G.D., 1997. Robust multiple-station magnetotelluric data processing, *Geophys. J. Int.,* **130**(2), 475–496.
Egbert, G.D., 2012. Hybrid conjugate gradient-Occam algorithms for inversion of multifrequency and multitransmitter EM data, *Geophys. J. Int.,* **190**(1), 255–266.
Egbert, G.D. & Kelbert, A., 2012. Computational recipes for electromagnetic inverse problems, *Geophys. J. Int.,* **189**(1), 251–267.
Eisenstat, S.C., Elman, H.C. & Schultz, M.H., 1983. Variational iterative methods for nonsymmetric systems of linear equations, *SIAM J. Numer. Anal.,* **20**(2), 345–357.
Falgout, R.D. & Yang, U.M., 2002. hypre: A library of high performance preconditioners, in *International Conference on Computational Science,* pp. 632–641, eds Sloot, P. M. A., Hoekstra, A. G., Tan, C. J. K. & Dongarra, J.J., Springer.
Grayver, A.V., 2015. Parallel three-dimensional magnetotelluric inversion using adaptive finite-element method. Part I: theory and synthetic study, *Geophys. J. Int.,* **202**(1), 584–603.
Grayver, A.V. & Bürg, M., 2014. Robust and scalable 3-D geo-electromagnetic modelling approach using the finite element method, *Geophys. J. Int.,* **198**(1), 110–125.
Grayver, A.V. & Kolev, T.V., 2015. Large-scale 3D geoelectromagnetic modeling using parallel adaptive high-order finite element method, *Geophysics,* **80**(6), E277–E291.
Grayver, A.V., Streich, R. & Ritter, O., 2013. Three-dimensional parallel distributed inversion of CSEM data using a direct forward solver, *Geophys. J. Int.,* **193**(3), 1432–1446.
Heagy, L.J., Cockett, R., Kang, S., Rosenkjaer, G.K. & Oldenburg, D.W., 2017. A framework for simulation and inversion in electromagnetics, *Comput. Geosci.,* **107,** 1–19.
Hiptmair, R. & Xu, J., 2007. Nodal auxiliary space preconditioning in H (curl) and H (div) spaces, *SIAM J. Numer. Anal.,* **45**(6), 2483–2509.
Kelbert, A., Meqbel, N., Egbert, G.D. & Tandon, K., 2014. ModEM: A modular system for inversion of electromagnetic geophysical data, *Comput. Geosci.,* **66,** 40–53.
Key, K., 2016. MARE2DEM: a 2-D inversion code for controlled-source electromagnetic and magnetotelluric data, *Geophys. J. Int.,* **207**(1), 571–588.
Key, K. & Weiss, C., 2006. Adaptive finite-element modeling using unstructured grids: The 2D magnetotelluric example, *Geophysics,* **71**(6), G291–G299.
Kolev, T.V. & Vassilevski, P.S., 2009. Parallel auxiliary space AMG for H (curl) problems, *J. Comput. Math.,* **27**(5), 604–623.
Kordy, M., Wannamaker, P., Maris, V., Cherkaev, E. & Hill, G., 2016. 3-D magnetotelluric inversion including topography using deformed hexahedral edge finite elements and direct solvers parallelized on SMP computers–Part I: forward problem and parameter Jacobians, *Geophys. J. Int.,* **204**(1), 74–93.
Liu, D.C. & Nocedal, J., 1989. On the limited memory BFGS method for large scale optimization, *Math. Program.,* **45**(1), 503–528.
Logg, A. *et al.*, 2012. *Automated Solution of Differential Equations by the Finite Element Method,* Springer.
Maaø, F.A., 2007. Fast finite-difference time-domain modeling for marine-subsurface electromagnetic problems, *Geophysics,* **72**(2), A19–A23.
Mackie, R.L. & Madden, T.R., 1993. Three-dimensional magnetotelluric inversion using conjugate gradients, *Geophys. J. Int.,* **115**(1), 215–229.
Mulder, W., 2006. A multigrid solver for 3D electromagnetic diffusion, *Geophys. Prospect.,* **54**(5), 633–649.
Myer, D., Constable, S. & Key, K., 2011. Broad-band waveforms and robust processing for marine CSEM surveys, *Geophys. J. Int.,* **184**(2), 689–698.
Newman, G.A., 2014. A review of high-performance computational strategies for modeling and imaging of electromagnetic induction data, *Surv. Geophys.,* **35**(1), 85–100.
Newman, G.A. & Alumbaugh, D.L., 2000. Three-dimensional magnetotelluric inversion using non-linear conjugate gradients, *Geophys. J. Int.,* **140**(2), 410–424.
Puzyrev, V., Koric, S. & Wilkin, S., 2016. Evaluation of parallel direct sparse linear solvers in electromagnetic geophysical problems, *Comput. Geosci.,* **89,** 79–87.
Qin, C., Wang, X. & Zhao, N., 2023. EMFEM: A parallel 3D modeling code for frequency-domain electromagnetic method using goal-oriented adaptive finite element method, *Comput. Geosci.,* **178,** 105403.
Qiu, C., Yin, C., Liu, Y., Ren, X., Chen, H. & Yan, T., 2021. Solution of large-scale 3D controlled-source electromagnetic modeling problem using efficient iterative solvers, *Geophysics,* **86**(4), E283–E296.
Rochlitz, R., Skibbe, N. & Günther, T., 2019. custEM: Customizable finite-element simulation of complex controlled-source electromagnetic data, *Geophysics,* **84**(2), F17–F33.
Rochlitz, R., Becken, M. & Günther, T., 2023. Three-dimensional inversion of semi-airborne electromagnetic data with a second-order finite-element forward solver, *Geophys. J. Int.,* **234**(1), 528–545.
Rodi, W. & Mackie, R.L., 2001. Nonlinear conjugate gradients algorithm for 2-D magnetotelluric inversion, *Geophysics,* **66**(1), 174–187.
Rücker, C., Günther, T. & Wagner, F.M., 2017. pyGIMLi: An open-source library for modelling and inversion in geophysics, *Comput. Geosci.,* **109,** 106–123.
Saad, Y., 1993. A flexible inner-outer preconditioned GMRES algorithm, *SIAM J. Sci. Comput.,* **14**(2), 461–469.
Si, H., 2015. Tetgen, a delaunay-based quality tetrahedral mesh generator, *ACM Trans. Math. Softw.,* **41**(2)..
Siripunvaraporn, W., 2012. Three-dimensional magnetotelluric inversion: an introductory guide for developers and users, *Surv. Geophys.,* **33,** 5–27.
Smirnova, M.V. *et al.*, 2019. A novel semiairborne frequency-domain controlled-source electromagnetic system: Three-dimensional inversion of semiairborne data from the flight experiment over an ancient mining area near Schleiz, Germany, *Geophysics,* **84**(5), E281–E292.
Streich, R., Becken, M. & Ritter, O., 2013. Robust processing of noisy land-based controlled-source electromagnetic data, *Geophysics,* **78**(5), E237–E247.
Toh, H., Schultz, A., Uyeshima, M. & Fujii, I., 2002. Three-dimensional modelling of the Earth's mantle, in *Proceedings of the Ocean Hemisphere Project 2,* Tokyo.
Usui, Y., 2015. 3-D inversion of magnetotelluric data using unstructured tetrahedral elements: applicability to data affected by topography, *Geophys. J. Int.,* **202**(2), 828–849.

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

Usui, Y., Ogawa, Y., Aizawa, K., Kanda, W., Hashimoto, T., Koyama, T., Yamaya, Y. & Kagiyama, T., 2017. Three-dimensional resistivity structure of Asama Volcano revealed by data-space magnetotelluric inversion using unstructured tetrahedral elements, *Geophys. J. Int.,* **208**(3), 1359–1372.

Wang, B., Liu, J., Hu, X., Liu, J., Guo, Z. & Xiao, J., 2021. Geophysical electromagnetic modeling and evaluation: a review, *J. Appl. Geophys.,* **194,** 104438.

Weiss, M., Neytcheva, M. & Kalscheuer, T., 2023. Iterative solution methods for 3D controlled-source electromagnetic forward modelling of geophysical exploration scenarios, *Comput. Geosci.,* **27**(1), 81–102.

Werthmüller, D., Mulder, W.A. & Slob, E.C., 2019. emg3d: A multigrid solver for 3D electromagnetic diffusion, *J. Open Source Softw.,* **4**(39), 1463.

Werthmüller, D., Rochlitz, R., Castillo-Reyes, O. & Heagy, L., 2021. Towards an open-source landscape for 3-D CSEM modelling, *Geophys. J. Int.,* **227**(1), 644–659.

Xu, J., 1996. The auxiliary space method and optimal multigrid preconditioning techniques for unstructured grids, *Computing,* **56**(3), 215–235.

APPENDIX A: AN OVERVIEW OF MODELLING CODES AND FORWARD SOLVERS IN EM MODELLING

A limited overview of modelling software in EM modelling, available as open source or on demand for academic purposes, and their employed forward solvers is given here.

Table A1. List of available solvers for various modelling codes in electromagnetic forward modelling. For completion, packages that possess inversion capabilities are indicated by ⇄. It is stressed that the listed solvers represent options for the forward problem. For solver information of the inverse problem, that is of the system of normal equation, the reader is referred to the respective publications.

Modelling code	direct solver(s)	iterative solver(s)
custEM (⇄, Rochlitz *et al.* 2019, 2023)	MUMPS	**Extended as part of this work:** Outer solver: Generalised conjugate residual (GCR) method preconditioned with the PREconditioner for Square Blocks (PRESB); Inner solver: Flexible generalised minimal residual (FGMRES) method with auxiliary-space Maxwell (AMS) preconditioner
EMFEM (Qin *et al.* 2023)	None	Outer solver: block-diagonal preconditioned FGMRES method; Inner solver: conjugate gradient (CG) method with AMS preconditioner
emg3d (Werthmüller *et al.* 2019)		Geometric multigrid method by Mulder (2006)
FEMTIC (⇄, Usui 2015; Usui *et al.* 2017)	Intel MKL PARDISO	None
GoFEM (⇄, Grayver 2015; Grayver & Kolev 2015)	MUMPS	Outer solver: block-diagonal preconditioned FGMRES method; Inner solver: conjugate gradient (CG) method with AMS preconditioner
MARE2DEM (⇄, Key & Weiss 2006; Key 2016)	Intel MKL PARDISO	
ModEM (⇄, Egbert & Kelbert 2012; Kelbert *et al.* 2014)		Quasi-minimum residual (QMR) method with an incomplete LU (ILU) factorization as preconditioner; A variant of biconjugate gradient (BiCGStab) method (after Toh *et al.* 2002)
PETGEM (Castillo-Reyes *et al.* 2018,2019)	MUMPS	GMRES solver preconditioned with either a successive over-relaxation (SOR), a geometric algebraic multigrid (GAMG), an additive Schwarz (ASM) or a Jacobi method; SOR preconditioned Transpose free quasi-minimal residual (TFQMR) method
SimPEG (⇄, Cockett *et al.* 2015; Heagy *et al.* 2017)	MUMPS; Intel MKL PARDISO	Biconjugate gradient stabilised (BiCGStab) method with Jacobi preconditioner

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

APPENDIX B: ACCURACY AND RESOURCE DEMANDS OF FORWARD SOLVERS

The accuracy of the direct and iterative solution is shown here. In addition, the measured time and memory demands used in Figs 5 and 6 are given as numbers.

B1 Accuracy comparison

A comparison between the responses for the direct and iterative solver for Marlim MR3D model is shown in Fig. B1 for the x-component of the electric field of the receiver line 04Tx013a (inline). The response is in excellent agreement as verified by the relative errors.

B2 Time and memory requirements

For completeness, the information portrayed in Figs 5 and 6 is represented as numbers given in Table B1.

Table B1. Solving times and memory requirements for the direct and iterative solvers for the five mesh discretizations of the MR3D model. From top to bottom, the times and memory demands are indicated for the simulations run with 4, 8, 16, 32 or 64 MPI processes.

	M1	M2	M3	M4	M5
			Direct solver		
Solving times (s)	25.3,	1056.7,	3595.1,	11,118.8,	14,747.3,
	18.5,	792.3,	4388.0,	11,916.3,	12,982.2,
	14.2,	466.3,	2895.1,	7656.0,	7611.3,
	15.3,	359.0,	1457.6,	3835.0,	4348.0,
	27.4	339.3	1106.3	3322.0	3777.1
Memory (GB)	5.9,	100.3,	289.0,	317.4,	332.8,
	7.9,	133.8,	361.7,	503.7,	536.5,
	10.8,	135.7,	477.3,	838.7,	961.5,
	19.8,	156.7,	462.9,	949.8,	1068.0,
	42.8	193.1	535.6	1044.5	1195.6
			Iterative solver		
Solving times (s)	24.7,	117.5,	235.1,	362.4,	503.1,
	13.2,	60.4,	122.3,	188.8,	248.4,
	7.7,	32.1,	65.6,	98.2,	137.0,
	5.1,	22.4,	36.5,	59.0,	79.4,
	5.0	16.6	25.7	44.0	53.0
Memory (GB)	1.38,	8.2,	15.9,	24.2,	31.6,
	1.38,	8.0,	15.6,	23.3,	31.2,
	1.37,	7.8,	15.2,	23.1,	32.2,
	1.15,	7.8,	15.9,	22.1,	29.4,
	1.01	7.8	15.4	23.7	30.9

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

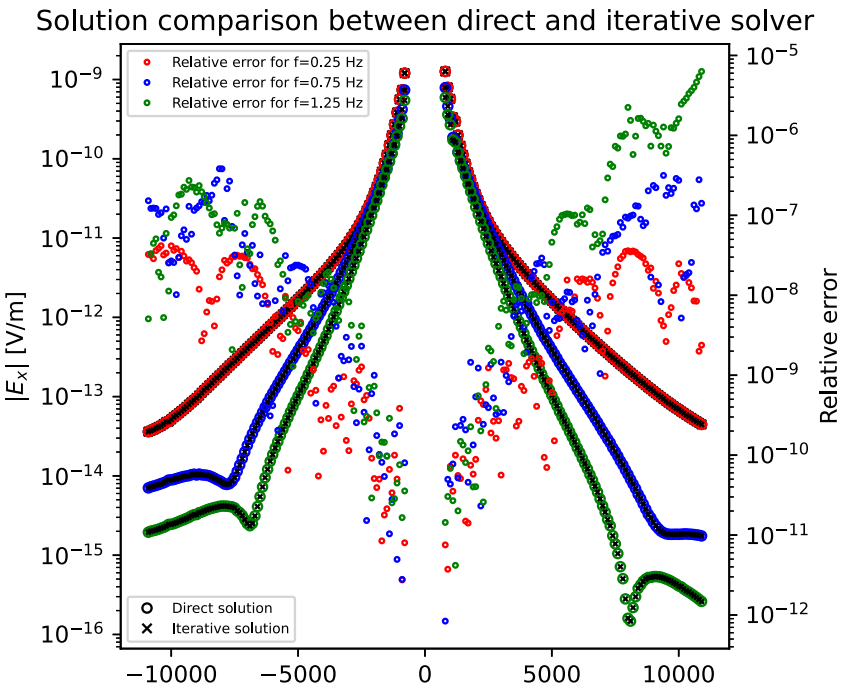


Figure B1. Amplitudes and the relative errors for the x-component of the electric field of the receiver line 04Tx013a (inline) are shown for three frequencies.

Downloaded from academic.oup.com/gji/article/243/1/ggaf290/8220040 by guest on 11 September 2026

© The Author(s) 2025. Published by Oxford University Press on behalf of The Royal Astronomical Society. This is an Open Access article distributed under the terms of the Creative Commons Attribution License (https://creativecommons.org/licenses/by/4.0/), which permits unrestricted reuse, distribution, and reproduction in any medium, provided the original work is properly cited.